Clase 2: DNNs y “un poco” de Álgebra Lineal

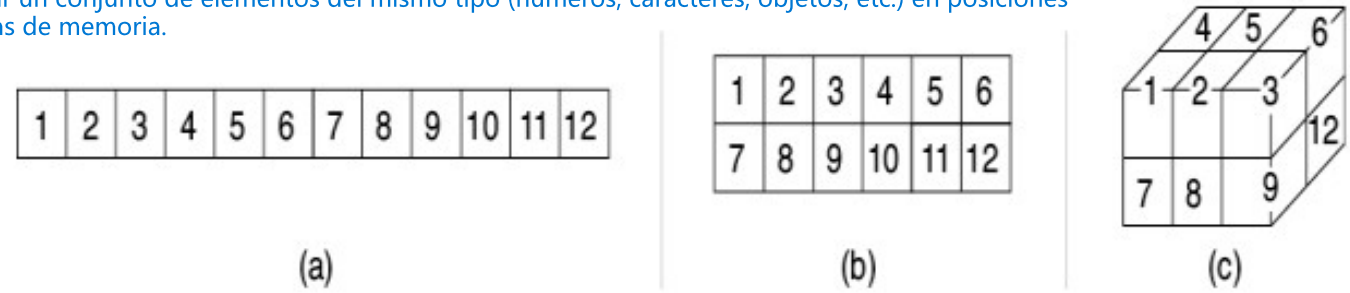
Tensores, dimensiones (batch/features/neuronas), producto escalar, composición de funciones. Backpropagation, regla de la cadena, gradientes (mismas dimensiones), forward/backward. Training loop, learning rate, épocas, arquitectura, overfitting.

Vectores, Matrices y tensores

Vector: Vector, en programación, es una estructura de datos que representa un arreglo unidimensional de elementos del mismo tipo, almacenados de forma contigua en memoria.

Matriz: una matriz (también llamada array o arreglo) es una estructura de datos que permite almacenar y organizar un conjunto de elementos del mismo tipo (números, caracteres, objetos, etc.) en posiciones contiguas de memoria.

Tensor:



Tensores en programación
En programación, un tensor se refiere a una estructura de datos que generaliza escalares, vectores y matrices a n dimensiones. Los tensores son fundamentales en el aprendizaje automático y la inteligencia artificial, ya que representan las entradas, salidas y el estado interno de una red neuronal. PyTorch, por ejemplo,

Matemática de las Redes Neuronales

Una red neuronal es una función matemática con multiples “inputs” y “outputs”

Es una función de funciones “anidadas” o “encadenadas”

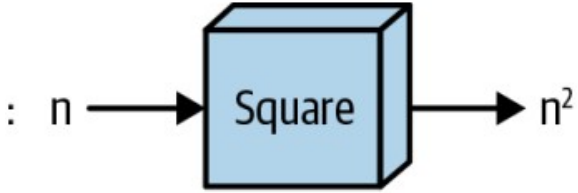
Puede ser representada por un grafo por el cual “fluyen” tensores (arreglos?)

La red se organiza en capas de neuronas (todas con el mismo tipo de función)

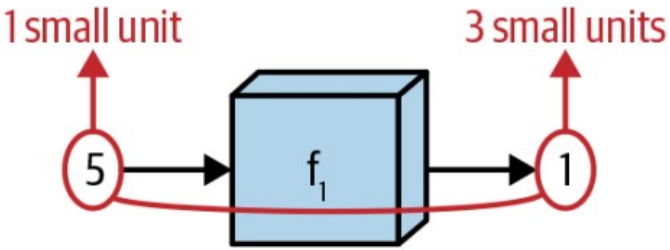
La red es un “aproximador” universal, o sea en teoría puede aproximar cualquier función.

Representaciones gráficas

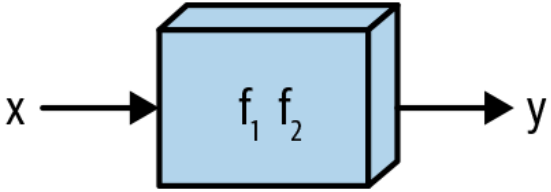
Función: $y=x^2$



Derivada: $dy/dx = 2x$



Funciones anidadas



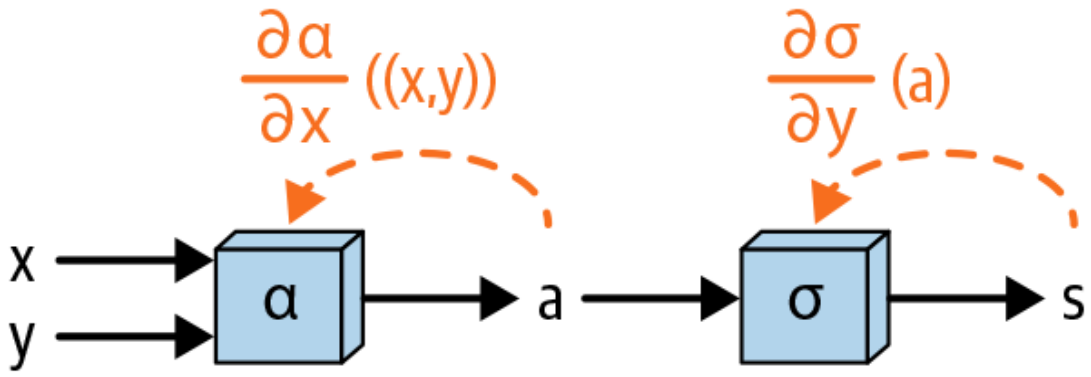
Regla de la cadena:

$df_2/dx(x) = df_2/dx\ (f_1(x)) * df_1/dx(x)$

Las funciones básicas de las redes neuronales

Función “alfa” es una suma pesada de múltiples inputs (x,y,z...)

Función “gama” es una función de activaciòn



Multiplicación de vectores y matrices

Input sea 1 fila de “c columnas” features (x1,x2,x3..xn), si tengo una sola “neurona” tengo los weights (w1, w2... wn) => Alfa = x1*w1 + x2*w2... +xn*wn => producto escalar (ot product) X*W

$$\begin{bmatrix} 4 & 3 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ 7 \\ 1 \end{bmatrix} = 4 * 2 + 3 * 7 + 6 * 1 = 35$$

Input son f filas => matriz f x c = [X], si aplico alfa sobre todas las filas obtengo como salida un vector de dimensión f filas (x 1 columna), lamados S, que es el resultado de la operación [X]*W = S

Agregar el “bias” es sumar al vector resultante una constante “b”

Multiplicación de vectores y matrices

Si en una capa tenemos “n” neuronas, en realidad W no es un vector sino una matriz de dimensiones c (columnas de features del input) x n (neuronas) [W], entonces $[X]_{fxc} * [W]_{cxn} = [S]_{fxn}$ donde ahora S es una matriz de dimensión fxn que contiene en cada fila el resultado de procesar ese input por la sn neuronas.

Veamos un ejemplo con “más” números...

Input (¿cuantos datos?)

$$X = \begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ x_{31} & x_{32} \end{pmatrix}$$

Neuronas (¿cuantas?)

$$W = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} \qquad b = \begin{pmatrix} b_1 & b_2 \end{pmatrix}$$

$$Z = XW + b = \begin{pmatrix} (x_{11} \cdot w_{11} + x_{12} \cdot w_{21}) + b_1 & (x_{11} \cdot w_{12} + x_{12} \cdot w_{22}) + b_2 \\ (x_{21} \cdot w_{11} + x_{22} \cdot w_{21}) + b_1 & (x_{21} \cdot w_{12} + x_{22} \cdot w_{22}) + b_2 \\ (x_{31} \cdot w_{11} + x_{32} \cdot w_{21}) + b_1 & (x_{31} \cdot w_{12} + x_{32} \cdot w_{22}) + b_2 \end{pmatrix}$$

$$Z = \begin{pmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \\ z_{31} & z_{32} \end{pmatrix}$$

¿que representa esta dimensión?

Y la FA?

La FA (i.e ReLU) se aplica elemento a elemento de Z ¿por qué?

ReLu = max(0,Z)

$$Z = \begin{pmatrix} 2.6 & -3.3 \\ -5.6 & 6.3 \\ 8.6 & -9.3 \end{pmatrix}$$

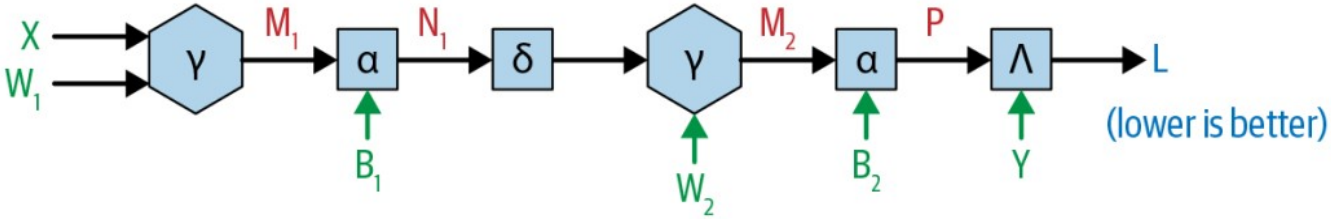
$$A = \mathrm{ReLU}(Z) = \begin{pmatrix} 2.6 & 0 \\ 0 & 6.3 \\ 8.6 & 0 \end{pmatrix}$$

¿que representa esta dimensión?

Red neuronal de 1 capa

1. Input X (matriz f x c) que es procesada por W1 (matriz de c pesos x n neuronas), que da como resultado M1 (f x n).
2. A cada valor de M1 se aplica un bias B1 => N1
3. Luego se aplica la función de activación “s”.
4. La matriz resultante (f x n) es ahora condensada en 1 output x fila utilizando W2 y aplicando bias 2 => esto da como resultado vector (dim f) que es la predicción (P) de cada input

Figure 2-10 is a diagram of what our more complicated model now looks like.



5. Para entrenar compar P con un vector de resultados conocido Y (RMSE, CBE) y obtengo 1 solo número que es el valor de la LF: L

“Backpropagation”

Recordemos la regla de la cadena:

Sean 2 funciones: $f1=3x+2$ y $f2 = f1^2$ ¿como es la derivada de f2 respecto de x?

$df2/dx = df2/df1 * df1/dx$

$df2/df1 = 2\ f1$ y $df1/dx = 3$ => $df2/dx = 2\ f1 * 3$

¿cuánto vale la derivada en x=1?

$df2/dx = 2\ f1(x=1) * 3 = 2 * (3*1 +2) *3 = 30!$

¿Cómo se relaciona con las RN?

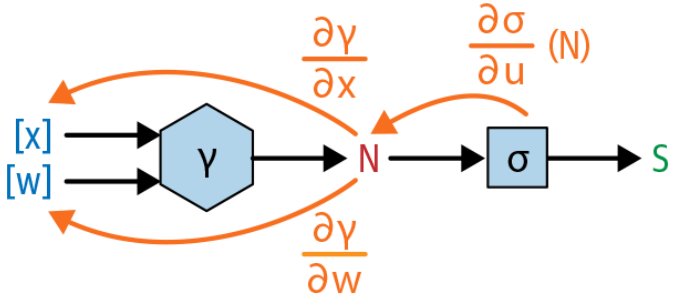
La derivada del producto escalar y la regla de la cadena

La derivada del producto escalar X+W respecto de cada elemento de W se puede demostrar que es equivalente a W transpuestas W^T :

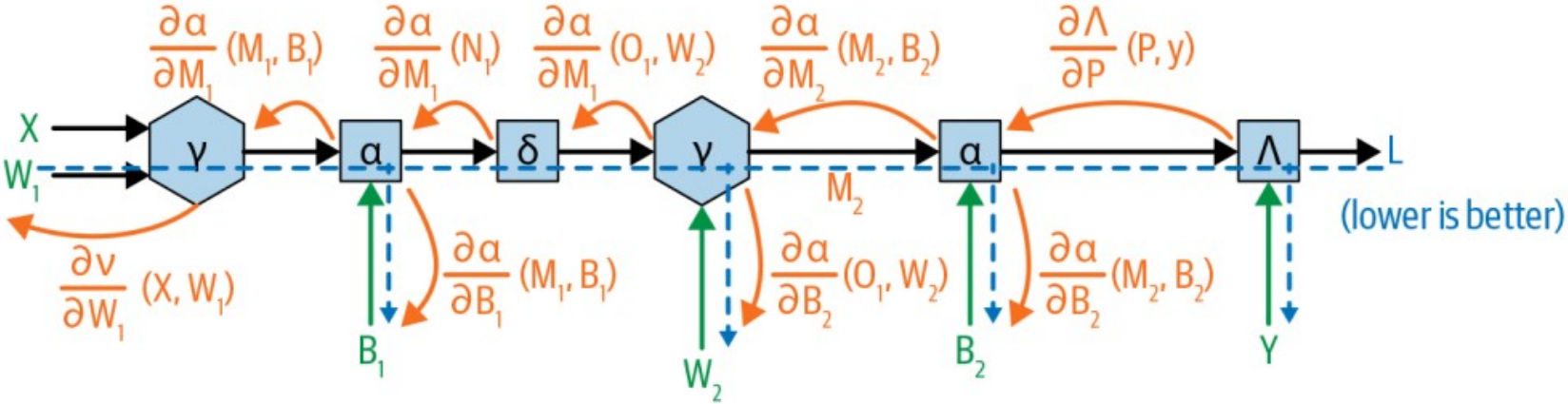
$$f = X * W \;\; => \;\; df/dw = W^T$$

Si ahora el resultado X * W = N, le aplicamos una función σ que da como resultado S por la regla de la cadena, el gradiente de S respecto de W es la derivada dσ/du (evaluada en el primer resultado N), multiplica por la derivada del producto escalar.

Note que para calcular dy/dx necesito saber cuanto vale ds/du(N)



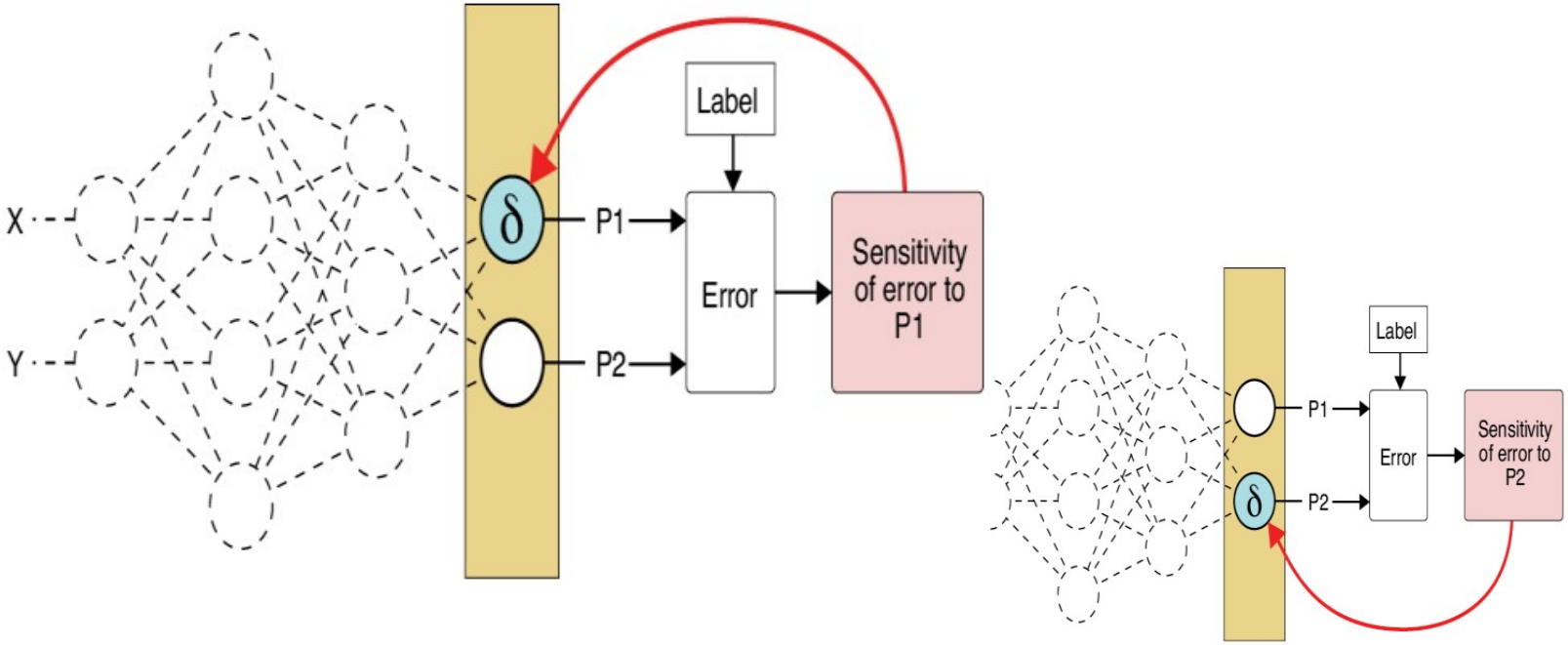
Y el gradiente respecto de “L”



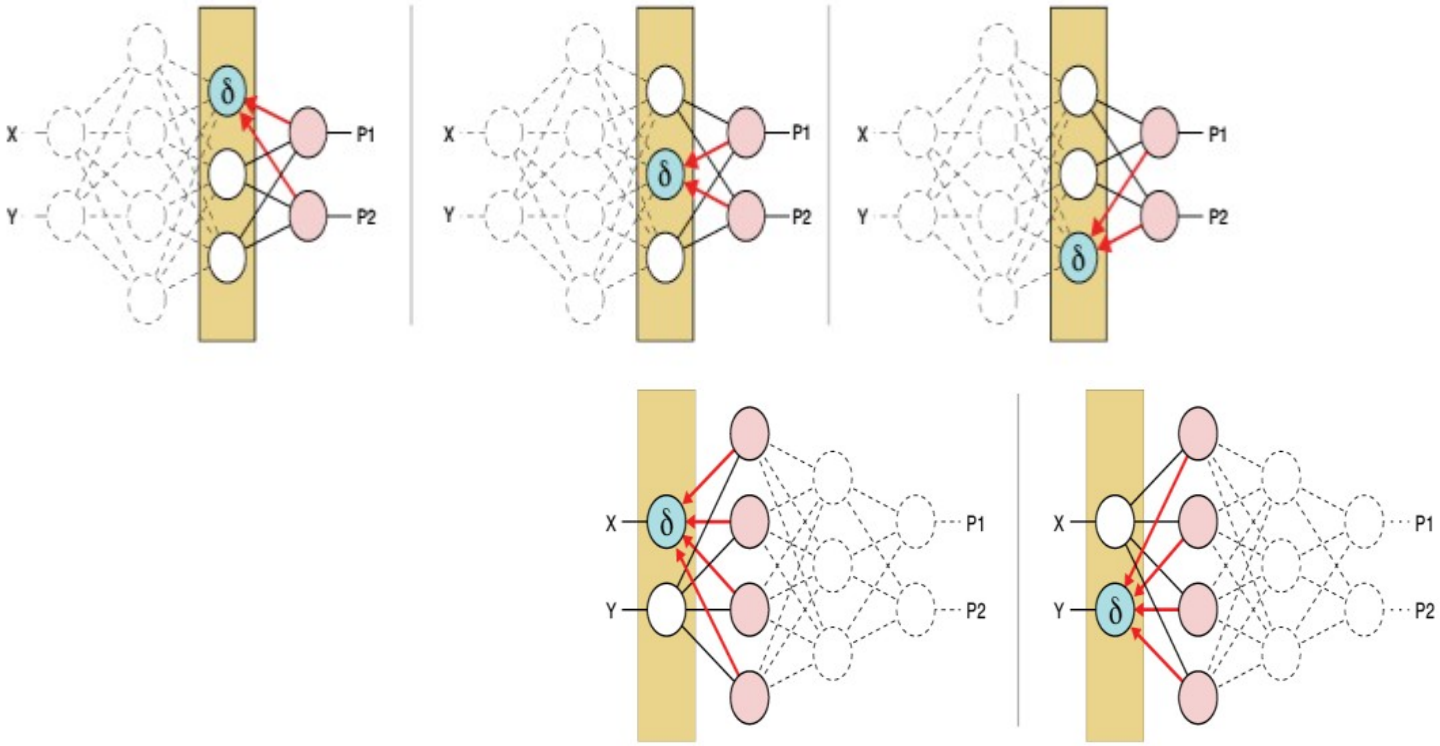
El fwd pass (azul) implica ir calculando las sucesivas matrices y vectores

El back pass (naranja) ir multiplicando las derivadas parciales evaluados con cada resultado parcial del fwd pass

Backprop “grafico”



Backprop “grafico”



En palábras...

El backpropagation nos permite calcular derivada (o gradiente) de cada parámetro de cada neurona en una red neuronal.

El mismo Depende de las derivadas de las neuronas de la capa siguiente, y en la “de salida” directamente a partir de la función de pérdida (que mide qué tan lejos está la predicción del valor real)

Una vez que hemos calculado el gradiente de todas las neuronas de una capa (incluyendo la capa de salida), podemos retroceder una capa hacia atrás y calcular los gradientes de todas las neuronas de esa capa anterior. Después retrocedemos otra vez, repetimos el proceso y seguimos así hasta llegar a la capa de entrada.

Cuando ya tenemos el delta de todas las neuronas de la red, usamos esa información para ajustar los pesos que conectan cada neurona con la capa anterior.

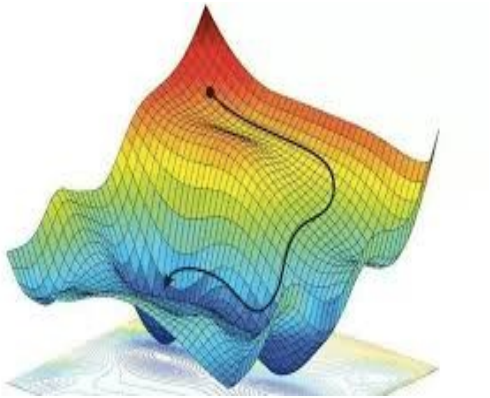
Interpretación geométrica del gradiente

Un vector que apunta en dirección de mayor incremento. ¿Cuáles son las dimensiones del gradiente?

Si W es (c x n), => ∇WL también es (c x n)

Input (X) (f × c) Weights (W) (c × n) Output (Z) (f × n)

$$\begin{bmatrix} x11 & x12 & \dots & x1c \\ x21 & x22 & \dots & x2c \\ . & . & . & \\ xf1 & xf2 & \dots & xfc \end{bmatrix} \times \begin{bmatrix} w11 & w12 & \dots & w1n \\ w21 & w22 & \dots & w2n \\ . & . & & . \\ wc1 & wc2 & \dots & wcn \end{bmatrix} = \begin{bmatrix} z11 & z12 & \dots & z1n \\ z21 & z22 & \dots & z2n \\ . & . & & . \\ zf1 & zf2 & \dots & zfn \end{bmatrix}$$



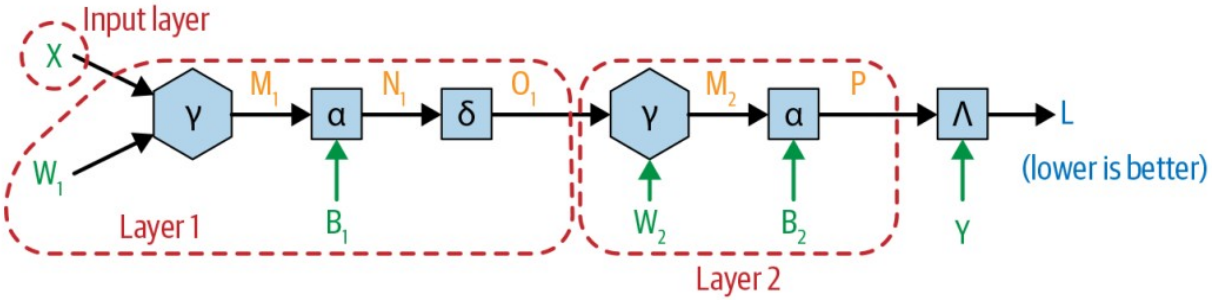
Gradiente respecto a W: W (c × n) entonces ∇W L (c × n)

$$\begin{bmatrix} w11 & w12 & \dots & w1n \\ w21 & w22 & \dots & w2n \\ . & . & . & \\ wc1 & wc2 & \dots & wcn \end{bmatrix} \rightarrow \begin{bmatrix} \partial L/\partial w11 & \partial L/\partial w12 & \dots & \partial L/\partial w1n \\ \partial L/\partial w21 & \partial L/\partial w22 & \dots & \partial L/\partial w2n \\ . & . & . & . \\ \partial L/\partial wc1 & \partial L/\partial wc2 & \dots & \partial L/\partial wcn \end{bmatrix}$$

∇WL indica cómo cambiar cada peso para reducir la loss, donde cada “elemento” dL/dwi mide la sensibilidad local de “L” a ese parámetro (wi)

Recordemos que entrenar: W_nuevo = W_viejo - dL/dwi *LR (LR = Learning Rate, el paso)

Comparando diagramas



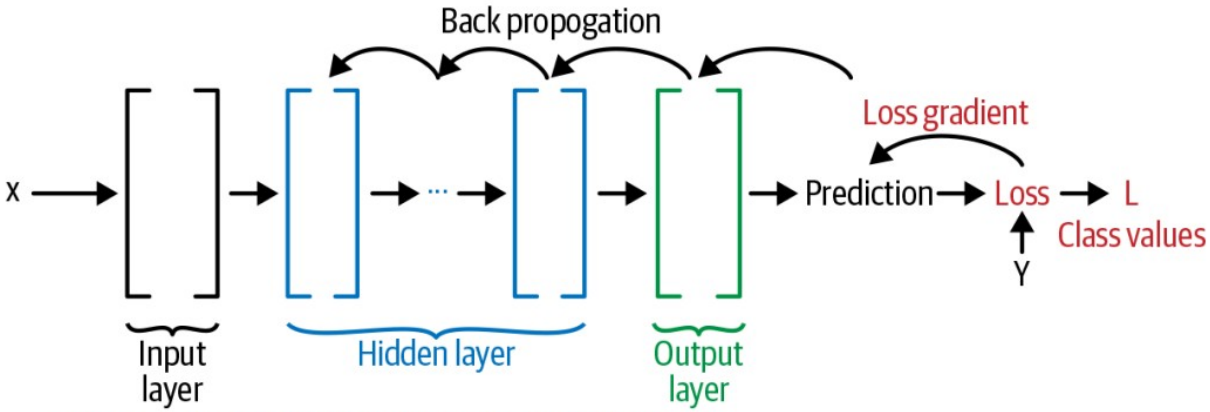


Figure 3-5. Backpropagation, now in terms of Layers instead of Operations

Resumiendo

Una “DNN” no es otra cosa que una serie de funciones “anidadas”

Poseen dos operaciones básicas “Producto escalar” y la “FA”

El “Backprop” implica ir calculando los gradientes hacia atrás usando derivadas encadenadas a partir de la LF.

Para “describirlas” usamos diagramas, que agrupan funciones y módulos reusables (Lego type)

¿ a ver si entendimos ?

- ¿Si tengo X(f×c) y W(c×n), qué representa cada dimensión y qué dimensión tiene la salida?
- ¿Qué significa que el gradiente tenga la misma dimensión que W? Y ¿Qué representa cada elemento del gradiente?
- ¿qué hace backpropagation sin usar fórmulas?
- ¿Cuál es la operación que mezcla información entre features en una red y por qué es importante?
- ¿Qué está haciendo realmente una capa de la red cuando calcula X·W? (pensarlo sin fórmulas)
- Si un peso wij tiene un gradiente grande, ¿qué me está diciendo eso sobre ese parámetro?
- ¿Por qué necesitamos guardar información del forward pass para hacer backpropagation?

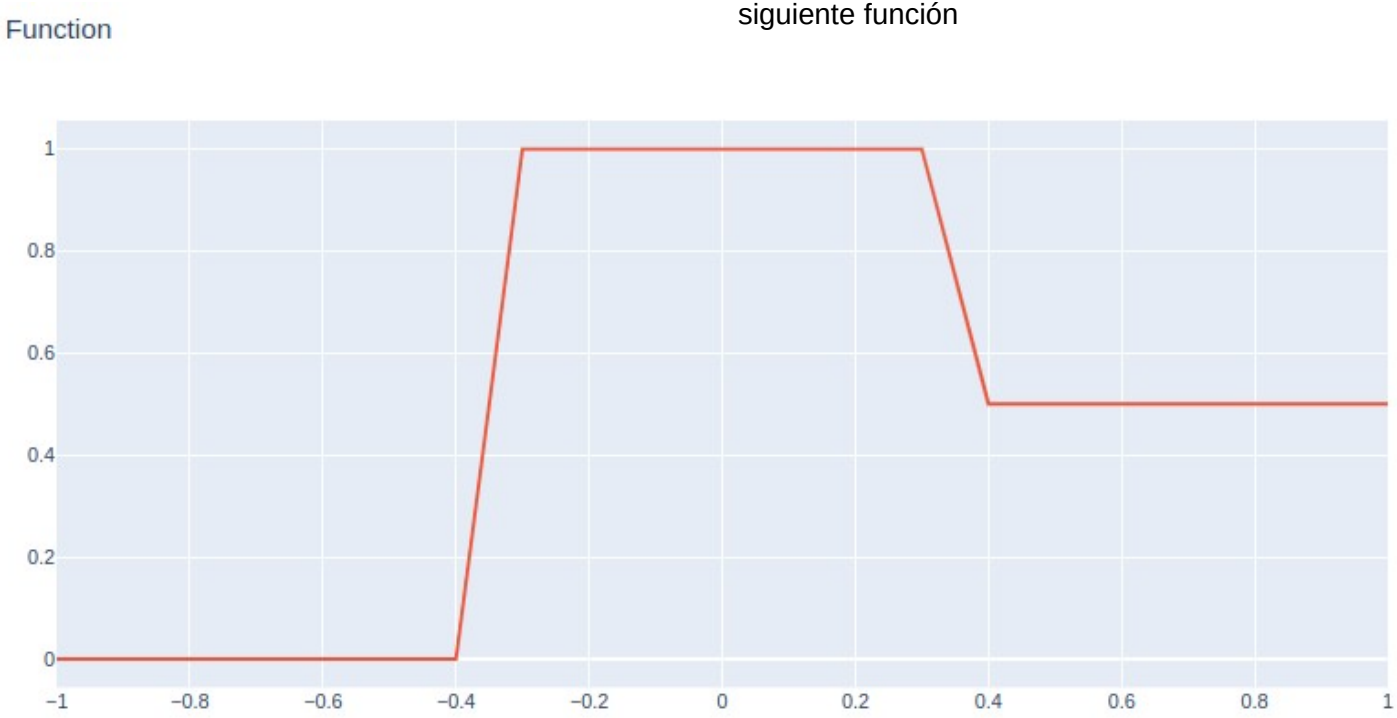
Y vamos al TP2

Tomando como punto de partida nuestra neurona artificial podemos construir pequeñas redes neuronales. La clave es entender que en el proceso de prediccion “forward” el output de una capa comprende la entrada de la capa siguiente, mientras que en el “backward pass” se va propagando con la regla de la cadena la derivada parcial hacia atras y se la evalúa en los “outputs” de las capas de los niveles previos.

El punto de partida es definir la arquitectura de la red, o sea cuantas capas, cuantas neuronas por capas y como es su conectividad. Si bien existe cierta flexibilidad al respecto, el problema a tratar usualmente coniciona el tipo de arquitectura a utilizar.

“Manos a la obra”

El objetivo de nuestra red será replicar la siguiente función



Ejericicio 2.0

Escriba un pequeño codigo que inicialice la red y le permita verificar su forma y la cantidad de weights y bias. Juegue alterando los valores de entrada hasta convencerse de que entiende como es inicializada la red.

Ejercicio 1.2.1) Escriba un codigo que dados los valores de entrada de la función (puede usar el intervalo -1 a 1, espaciado en 0.1), lo que serìa un batch, realice el fwd_pass para una dada red inicializada al azar y grafique el resultado. Grafique ademas la función correcta. Pruebe diferentes arquitecturas. ¿cree que es posible solo por azar obtener una red que recupere la función?

Ejercicio 2.1)

Escriba un codigo que tomando los valores de entrada de la función (use el intervalo -1 a 1, espaciado en 0.1), realicé el entrenamiento y permita analizar (graficar) como decae la LF , y luego como es la predicciòn de la red entrenada comparada con la real.

Analicé el efecto de la cantidad de pasos y el LR (consejo, pruebe utilizar un LR que decaiga a medida que avanza el entrenamiento). Analicè el efecto de cambiar la arquitectura de las capas ocultas de la red (recuerde que necesita 1 input y 1 neurona de salida).

Nota: recuerde que los ipnuts y valores y_true deben tener formato np.array (es muy fàcil convertir una lista a este formato).

Nota: recuerde que para entrenar a la red un ciclo (epoca) se debe i) realizar el fwd pass para obtener las activaciones, incluida la salida final con la que se cacula la LF, y con esa información ii) realizar el backprop para determinar los gradientes, que son utilizados para actualizar los parámetros (w,b)

Ejercicio 2.3) Ajuste de hiperparàmetros

Empecemos por analizar el LR y #E, para eso usaremos una arquitectura fija de 3 capas ocultas con 3 neurona cada una (puede probar con otras si quiere)

a) Analicé como evoluciona la LF y la función predicha a medida que aumenta el #E (trate de elegir un LR que de resultados significativos punto b). ¿que pasa si utiliza pocos pasos? ¿que pasa si utiliza demasiados?

b) Analicé como evoluciona la LF y la función predicha con diferentes LR (empiece en un valor chico 0.1-1 e incremente de a 2-10x). ¿puede encontrar un valor (rango) de LR optimo?

Como en cualquier optimización en general a medida que uno se acerca al mínimo se deben hacer pasos más pequeños. Pruebe implementar un LR que vaya decreciendo ¿còmo afecta el mismo a la LF y a la función resultante?

c) Analicé (utilizando un #E y LR optimos) el resultado de la LF y función predicha para diferentes arquitecturas (variando la capas ocultas). Comience con 1 capa de 1 neurona (1), luego incremente en el número de neuronas (2), o en el número de capas (1,1), o ambas (2,2). Etc, puede incluso avanzar a redes más complejas como (2,3,5,3,2) etc. ¿hay algun umbral mínimo de neuronas o capas necesario para reproducir la funciòn razonablemente? ¿qué pasa si el número es menor? ¿que pasa si aumentamos demasiado la complejidad?

Habiendo realizado el análisis propuesto, que puede decir del termino “overfitting”.